

万维网

理论课程

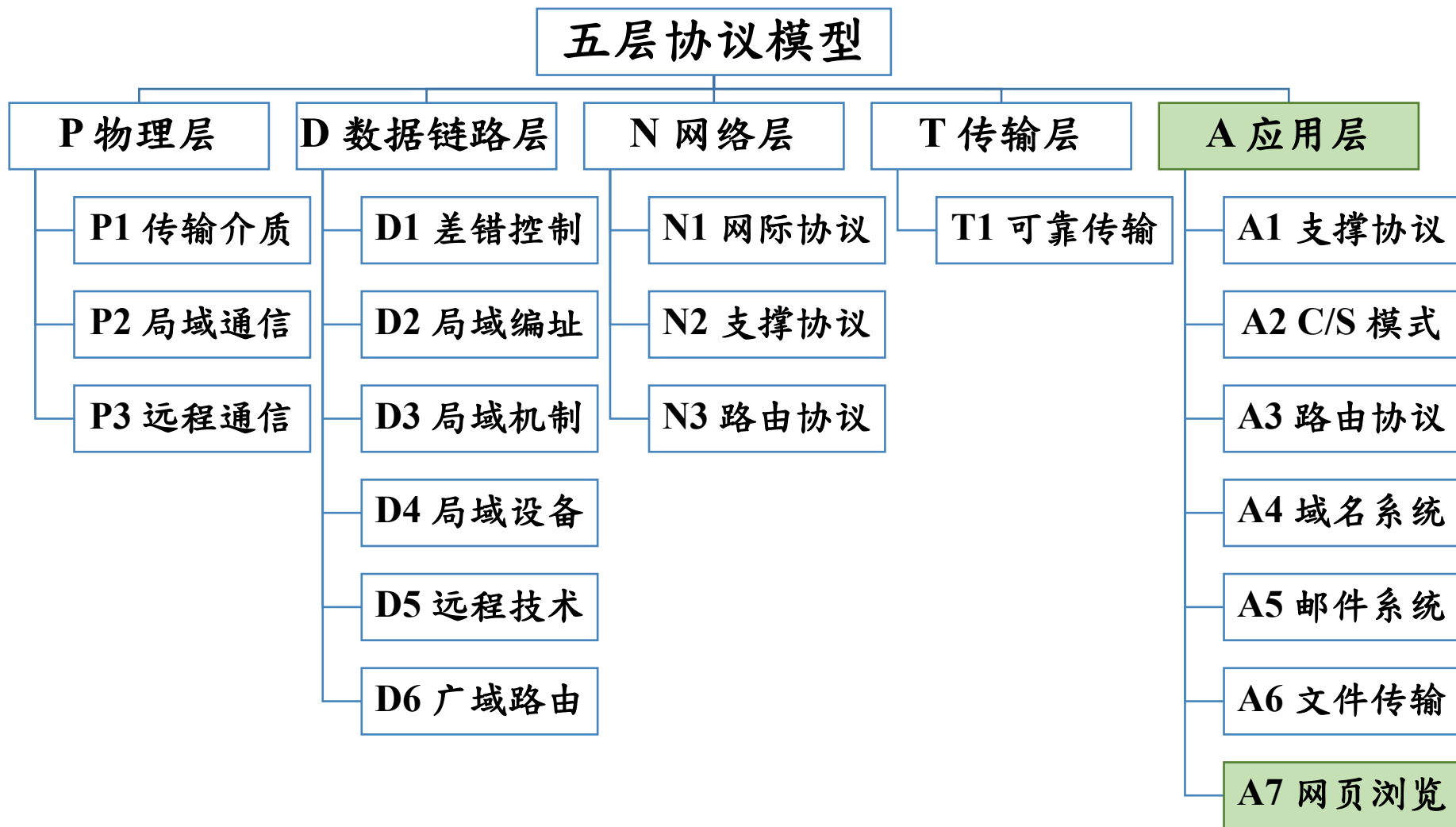


廈門大學
XIAMEN UNIVERSITY



信息学院 黄 焯
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang

知识框架



主要内容

- HTTP

- 工作原理与过程
- 错误代码
- URL
- HTML文档

对应课本章节

- **PART I Introduction And Internet Applications**
 - **Chapter 4 Traditional Internet Applications**
 - **4.3~4.9 Representation And Transfer; Web Protocols; Document Representation With HTML; Uniform Resource Locators And Hyperlinks; Web Document Transfer With HTTP; Caching In Browsers; Browser Architecture**

内容纲要

1	概述
2	HTTP 协议
3	客户端与服务端
4	Web 安全
5	性能优化

万维网的由来

- 互联网存在30年，但用户和用途非常有限
 - 用户：主要是大学、军事机构、科研单位的研究人员。
 - 用途：远程登录（Telnet）、文件传输（FTP）、电子邮件、新闻组等。所有操作都需要记住命令和地址，比如要通过ftp命令登录某台服务器，然后cd切换目录、get下载文件。
- 遇到的问题
 - 研究人员使用的计算机、操作系统和文件格式可能不同。
 - 用户需要频繁分享文档、实验数据、研究成果，但没有一个统一、简单的方式让所有人跨平台访问信息。
 - 即使信息已经存在某台服务器上，其他人也很难找到它。

万维网的由来

- 核心设计目标

- 统一的信息访问方式：不管文档在哪个服务器、用什么格式，都能通过同一个“工具”（浏览器）查看。
- 超链接：把相关信息连接起来，不需记住地址，只需点击。
- 简单、开放的协议：任何人都可以搭建，不需要特别许可。

- 万维网（World Wide Web，WWW）

- 万维网是运行在互联网之上的一个全球性信息系统，由互相链接的文档（网页）和其他资源（如图片、视频）组成。
- 平时用浏览器打开的“网站”整体，就是万维网。

万维网的概念

- 网页 (Web Page)

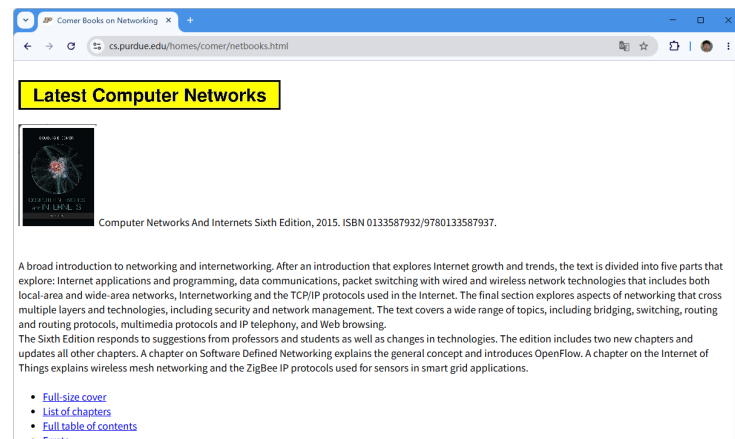
- 万维网上的一个文档，通过浏览器显示其内容。

- 网站 (Website)

- 由多个相关的网页组成的集合，存放在一台或多台服务器上，通常拥有一个共同的域名。

- 浏览器 (Browser)

- 安装在用户计算机上的客户端软件，用于请求、解析并显示网页。



万维网的概念

- **Web 服务器 (Web Server)**
 - 一台存储网页并提供服务的计算机 (或程序) ，它能接收来自浏览器的请求，并返回所请求的网页内容。
- **资源 (Resource)**
 - 可以通过特定字符串访问并获取的任何信息对象 (如：网页、图片) 。
- **统一资源定位符 (URL)**
 - 唯一标识万维网上每个资源的地址字符串。

万维网的概念

- 超文本传输协议 (HTTP)
 - 浏览器与 Web 服务器之间传输数据的通信规则 (协议) 。
它定义了请求和响应的格式。
- 超文本标记语言 (HTML)
 - 用于编写网页的标准语言。
 - 通过“标签” (如 <h1>、<p>) 描述网页的结构和内容。
- 超链接 (Hyperlink)
 - 网页中可点击的元素 (文字或图片) ，点击后将跳转到另一个网页或资源。

万维网的概念

- 超文本 (Hypertext)

- 一种非顺序访问的文本信息组织方式。
- 它通过链接 (link) 将不同位置的文本片段相互连接
- 读者可以根据需要，点击链接跳转到相关的内容，而不是像传统书籍那样必须一页一页顺序阅读。

- 多媒体 (Multimedia)

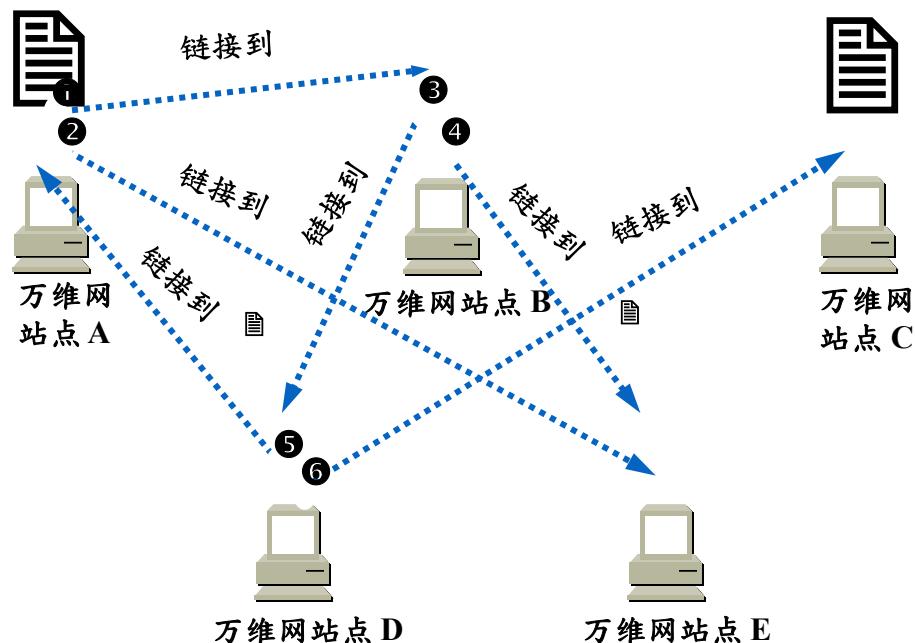
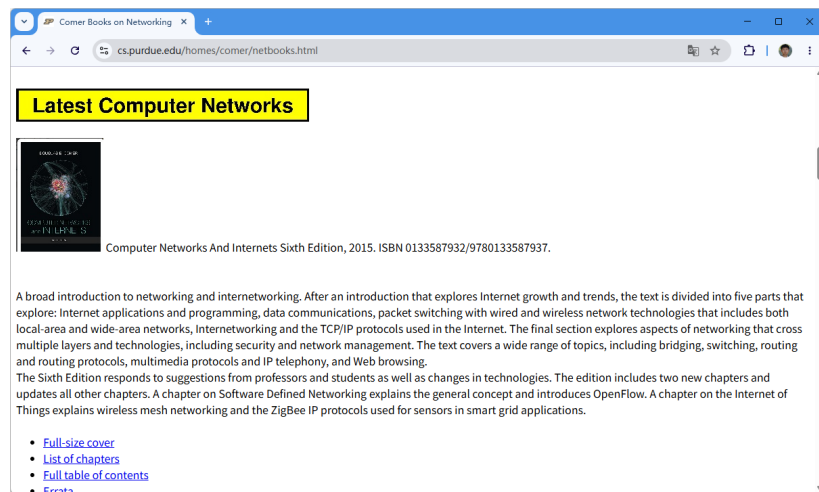
- 图形、图像、声音、动画、视频等

- 超媒体 (Hypermedia)

- 超媒体是超文本对多媒体的扩展，可以链接多媒体

万维网的概念

- 万维网通过超链接将不同的资源连接起来



万维网的工作方式

- 万维网以客户-服务器方式工作。
- 浏览器就是在用户计算机上的万维网客户程序。
- 万维网文档所驻留的计算机则运行服务器程序，因此这个计算机也称为万维网服务器。
- 客户程序向服务器程序发出请求，服务器程序向客户程序送回客户所要的万维网文档。

万维网必须解决的问题

- 怎样标识分布在整个因特网上的万维网文档？
 - 使每个文档在整个因特网的范围内具有唯一的标识符 URL。
- 用何协议实现万维网上各种超链的连接？
 - HTTP，使用 TCP 连接进行可靠的传送。
- 怎样使各种文档在各种计算机显示并找到超链位置？
 - 超文本标记语言 HTML (Hyper-Text Markup Language)
- 怎样使用户能够很方便地找到所需的信息？
 - 使用各种的搜索工具（即搜索引擎）。

万维网发展的时间线

- 1969 : ARPANET诞生，互联网的雏形
- 1974 : TCP/IP 协议为不同网络互联提供通用语言
- 1983 : TCP/IP 成为标准协议，现代互联网正式诞生
- 1985 : DNS 普及，无需再记数字 IP
- 1991 : 第一个网页和 Web 服务器，万维网诞生
- 1993 : Mosaic 浏览器出现，普通用户也能上网
- 1994 : Netscape 浏览器发布，W3C 成立
- 1995 : Windows 95 内置 TCP/IP 和 Internet Explorer

内容纲要

1	概述
2	HTTP 协议
2.1	HTTP 概述
2.2	统一资源定位符
2.3	HTTP 消息结构

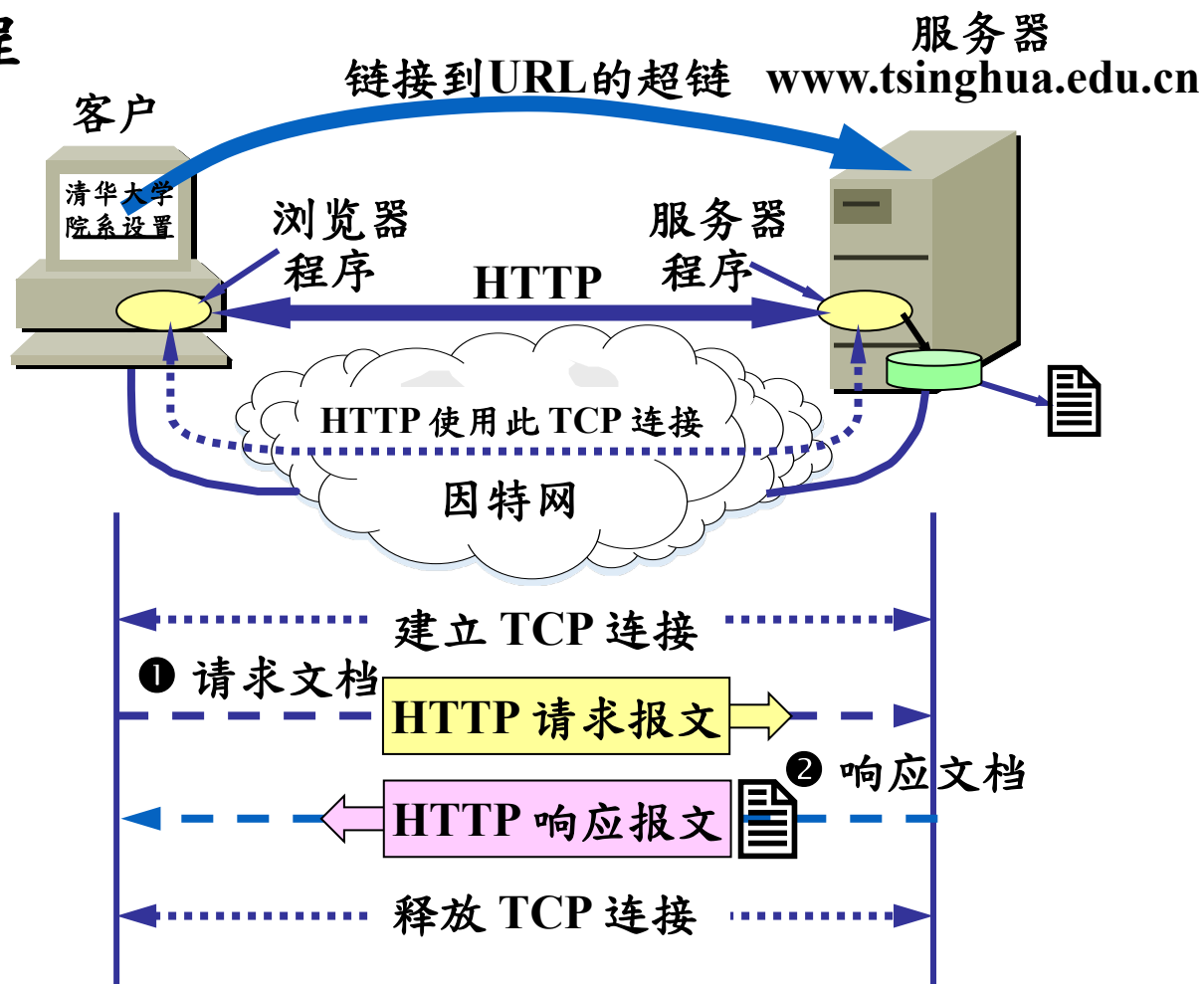
超文本传送协议 HTTP

- 超文本传输协议（HTTP）是一种应用层协议
 - 全称：HyperText Transfer Protocol
 - 专用于在浏览器和服务端之间传输超文本和其他网络资源。
 - 它是万维网（WWW）数据通信的基础。
- 核心特点
 - 无状态性：请求是独立的，服务器不会记住先前请求信息
 - 基于TCP：建立在TCP协议之上，通常使用80端口
 - 明文传输：数据以明文形式传输，不进行加密
 - 请求-响应模型：客户端发起请求，服务器返回响应

超文本传送协议 HTTP

• HTTP 的操作过程

- 浏览器发送请求
- 服务器返回响应
- 浏览器解析响应内容并渲染页面



HTTP 版本

版本	发布时间	主要改进
HTTP/0.9	1991年	最简版本，仅支持GET方法
HTTP/1.0	1996年	支持多种方法、状态码、MIME类型
HTTP/1.1	1999年	持久连接、管道化、虚拟主机（当前最广泛使用）
HTTP/2	2015年	二进制传输、多路复用、头部压缩
HTTP/3	2022年	基于QUIC协议，解决队头阻塞问题

用户点击鼠标后所发生的事件

- 浏览器分析超链指向页面的 URL 。
- 浏览器向DNS请求解析www.tsinghua.edu.cn的IP地址 。
- 域名系统DNS解析出清华大学服务器的 IP 地址 。
- 浏览器与服务器建立 TCP 连接
- 浏览器发出取文件命令：GET /chn/yxsx/index.htm 。
- 服务器给出响应，把文件 index.htm 发给浏览器 。
- TCP 连接释放 。
- 浏览器显示院系设置文件 index.htm 中的所有文本 。

内容纲要

	2	HTTP 协议
	2.1	HTTP 概述
	2.2	统一资源定位符
	2.3	HTTP 消息结构
	2.4	传输机制

统一资源定位符 URL

- 统一资源定位符 (Uniform Resource Locator , URL)
 - 格式 : `<协议>://<主机>:<端口>/<路径>?<查询>#<片段>`
 - 示例 : `https://myoj2.vscode.live:20102/OJ/problemset.html`
`ftp://ftp.belnet.be/mirror/ubuntu.com/lis-lR.gz`
- URL 是对因特网资源的**位置**和**访问方法**的简洁表示。
 - 不关心资源内部如何存储，只关注在哪里、如何访问。
 - 只要能够对资源定位，系统就可以对资源进行各种操作，如存取、更新、替换和查找其属性。
- URL 相当于一个文件名在网络范围的扩展。

统一资源定位符 URL

• URL示例

```
https://myoj2.vscode.live:20102/OJ/problem.html?id=1000#problem-hint
```

部分	含义	示例值	必须
协议	访问方式	https	必须
主机	域名或IP	myoj2.vscode.live	必须
端口	服务器端口（默认可省略）	20102	可选
路径	资源位置	/OJ/problem.html	可选
查询	动态参数（键值对）	id=1000	可选
片段	页面内锚点	problem-hint	可选

内容纲要

	2	HTTP 协议
	2.2	统一资源定位符
	2.3	HTTP 消息结构
	2.4	传输机制
	3	客户端与服务端

HTTP 的报文结构

- HTTP 有两类报文：
 - 请求报文：从客户向服务器发送请求。
 - 响应报文：从服务器到客户的回答。
- 报文中的每一个字段都是字符串，长度是不确定的。
- 报文由三个部分组成
 - 起始行
 - 请求报文：请求行
 - 响应报文：状态行
 - 头部字段
 - 消息体

起始行 (Start Line)

头部字段 (Header Fields)

空行 (CRLF , 即 \r\n)

消息体 (Message Body , 可选)

HTTP 的报文结构

- 请求报文，起始行即为请求行。
 - 格式：方法 空格 请求目标 空格 HTTP版本 回车换行
 - 方法大写，请求目标不包含协议和域名，回车换行应为\r\n
 - 示例： GET /index.html HTTP/1.1\r\n
- 响应报文，起始行即为状态行。
 - 格式：HTTP版本 空格 状态码 空格 原因短语 回车换行
 - 示例： HTTP/1.1 200 OK
- 头部字段
 - 格式：字段名 英文冒号 空格 字段值
 - 示例： Content-Type: text/html; charset=utf-8

HTTP 的报文结构

HTTP 请求	说明
GET /index.html HTTP/1.1	请求行
Host: example.com	头部字段
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36\r\n	
	空行，以下无数据

HTTP 响应	说明
HTTP/1.1 200 OK	状态行
Content-Type: text/html	头部字段
Content-Length: 1024	
	空行
<html>...</html>	消息体（实际的HTML）

HTTP 方法 (Methods)

- HTTP 方法说明用户对资源执行的操作。

方法 (大写)	作用	安全	幂等	有主体
GET	获取资源	✓ 是	✓ 是	✗ 否
HEAD	只获取头部，不返回主体	✓ 是	✓ 是	✗ 否
POST	提交数据，创建或处理	✗ 否	✗ 否	✓ 是
PUT	替换整个资源	✗ 否	✓ 是	✓ 是
PATCH	部分修改资源	✗ 否	✗ 否	✓ 是
DELETE	删除资源	✗ 否	✓ 是	✗ 否
CONNECT	建立隧道 (用于代理)	-	-	-
OPTIONS	询问服务器支持哪些方法	✓ 是	✓ 是	✗ 否
TRACE	回显请求 (调试用)	✓ 是	✓ 是	否
安全：不会改变服务器状态。		幂等：多次执行结果相同。		

HTTP 状态码 (Status Codes)

- 状态码是服务器表示对请求处理结果的三位数字。
 - 第一位数字 (百位数) 定义了响应的类别，共五类。
 - 后两位数字 没有分类作用，仅用于细分具体状态。

类别	首位	说明	示例
信息性	1xx	请求已接收，正在继续处理。	100 Continue
成功	2xx	请求已被成功接收、理解并接受。	200 OK
重定向	3xx	需要进一步操作才能完成请求，通常用于跳转或缓存。	304 Not Modified
客户端错误	4xx	请求包含语法错误或服务器无法满足。	404 Not Found
服务器错误	5xx	服务器在处理有效请求时发生错误。	500 Internal Server Error

HTTP 状态码

状态码	官方定义
100 Continue	客户端可以继续发送请求。
200 OK	请求成功。响应的内容取决于请求的方法。
201 Created	请求已完成，并创建了新的资源。
204 No Content	服务器已处理请求，但无内容返回（且无消息体）。
301 Moved Permanently	请求的资源已永久移动到新 URL。
302 Found	请求的资源临时移动到新 URL。
304 Not Modified	条件性请求满足，资源未修改，可直接使用缓存。
400 Bad Request	服务器无法理解请求语法。
401 Unauthorized	请求需要用户认证（缺乏凭证）。
403 Forbidden	服务器理解请求但拒绝执行（权限不足）。
404 Not Found	服务器找不到请求的资源。
405 Method Not Allowed	请求方法不被该资源支持。
500 Internal Server Error	服务器遇到意外情况，无法完成请求。
502 Bad Gateway	网关或代理收到来自上游服务器的无效响应。
503 Service Unavailable	服务器当前无法处理请求（过载或维护）。

HTTP 状态码

- 错误原因相同（如页面无法找到），代码可能不同
 - 左图：服务器处理的重定向（HTTP 302）
 - 右图：服务器美化的错误页面（HTTP 404）



头部字段 (Header Fields)

- 头部字段是键值对，用于传递附加信息。
 - 位于起始行之后、空行之前。
 - 格式：`key: value`

类别	作用	常见字段
通用头	同时适用于请求和响应	Date、Cache-Control、Connection
请求头	只有请求中使用	Host、User-Agent、Accept、Cookie、Authorization
响应头	只有响应中使用	Server、Set-Cookie、Location、WWW-Authenticate
实体头	描述消息体	Content-Type、Content-Length、Content-Encoding、Content-Language

HTTP表述 (Representation)

- 表述是指服务器实际返回给客户端的资源的具体形式，以及客户端发送给服务器的数据的具体格式。
 - 内容协商：客户端通过 **Accept** 头部告诉服务器它希望收到什么格式；服务器通过 **Content-Type** 告诉客户端它实际返回了什么格式。
 - 表述元数据：**Content-Type**、**Content-Encoding**（如 **gzip**）、**Content-Language**（如 **zh-CN**）。

```
GET /user/5 HTTP/1.1
Host: api.example.com
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 56
```

```
{"id":5, "name":"张三"}
```

HTTP缓存 (Caching)

- 缓存机制

- 允许客户端 (或中间代理) 保存已获取的响应，下次请求同一资源时直接使用缓存，而不用再向服务器请求。
- 这可以显著减少网络延迟和服务器负载。

缓存类型	原理	关键头部
强缓存	在有效期内直接使用缓存，完全不请求服务器	Cache-Control: max-age=3600 Expires
协商缓存	需要向服务器确认缓存是否仍然有效 如果没有变化，返回：304 Not Modified	ETag / If-None-Match Last-Modified / If-Modified-Since

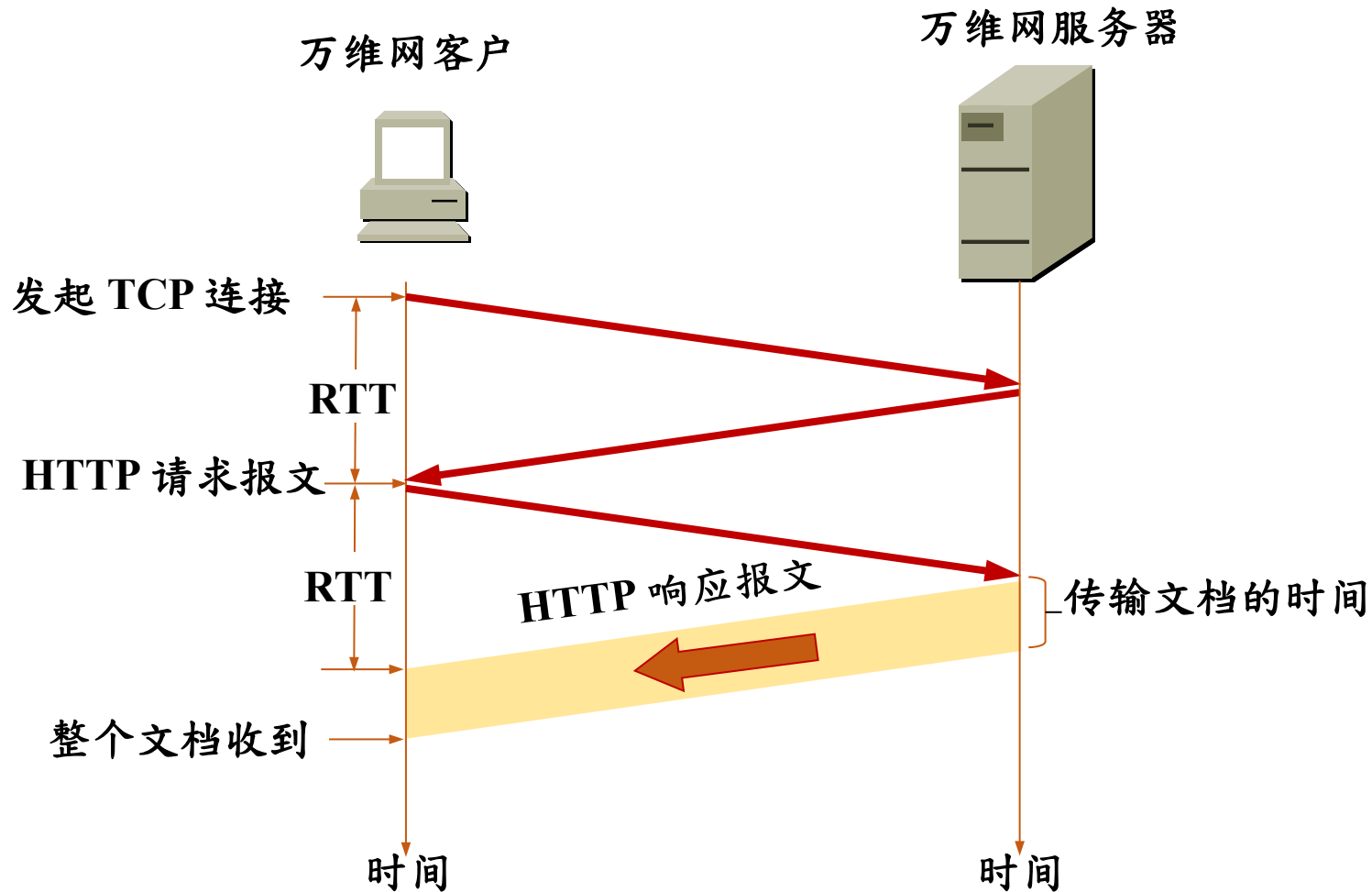
HTTP认证 (Authentication)

- HTTP 提供了简单的认证框架
 - 服务器能够要求客户端证明身份，决定是否允许访问资源。
- 核心流程 (挑战-响应)
 - 客户端请求受保护的资源 (如 /admin) 。
 - 服务器返回 401 Unauthorized，并在 WWW-Authenticate 头部中说明认证方式 (如 Basic 或 Bearer) 。
 - 客户端重新发送请求，在 Authorization 头部中附上凭证 (如用户名/密码的 Base64 编码或 Token) 。
 - 服务器验证凭证，通过则返回资源；失败则再次返回 401 。

内容纲要

	2	HTTP 协议
	2.3	HTTP 消息结构
	2.4	传输机制
	3	客户端与服务端
	4	Web 安全与性能优化

请求一个万维网文档所需的时间



持续连接 (persistent connection)

- HTTP/1.1 协议使用持续连接。
 - 服务器在发送响应后仍然在一段时间内保持这条连接，使同一个客户（浏览器）和该服务器可以继续在该连接上传送后续 HTTP 请求报文和响应报文。
 - 只要这些文档都在同一个服务器上就行。
- HTTP/1.1的流水线方式
 - 在持续连接的基础上，客户端不必等待前一个响应，而连续发送多个请求。服务器收到请求后，按顺序返回响应。
 - 已被 HTTP/2 多路复用取代，允许乱序返回响应。

在服务器上存放用户的信息

- HTTP 是无状态的协议，不记住用户信息。
- 万维网站点使用 Cookie 和 Session 来跟踪用户。
- Cookie 是服务器发送给浏览器的一小段文本信息，浏览器会将其保存，并在后续请求中自动携带回服务器。
 - 使用 Cookie 的网站服务器为用户产生一个唯一的识别码。利用此识别码，网站就能够跟踪该用户在该网站的活动
- 会话（Session）是服务器端为每个用户创建的独立存储空间，用于保存该用户在访问期间的状态信息。
 - 每个 Session 有一个唯一的 Session ID，信息位于服务器端

Cookie

- 特点

- 存储在客户端（浏览器）
- 大小受限（通常 4KB）
- 可以设置过期时间（持久 Cookie 存在硬盘，会话 Cookie 存在内存）
- 有隐私风险（可能被篡改或泄露）

- 工作流程

- 服务器通过 Set-Cookie 响应头下发 Cookie。
- 浏览器保存 Cookie。
- 之后每次请求，浏览器通过 Cookie 请求头自动携带。

会话 (Session)

- 特点

- 存储在服务器端 (内存或数据库)
- 容量无严格限制 (取决于服务器)
- 每个 Session 有一个唯一的 Session ID
- 更安全 (用户无法直接查看或修改 Session 内容)

- 工作流程

- 用户首次访问，服务器创建 Session，生成唯一 Session ID。
- 服务器将 Session ID 通过 Cookie 发送给浏览器。
- 浏览器后续请求携带该 Cookie。
- 服务器根据 Session ID 找到对应的 Session 对象，读写数据。

内容纲要

	2	HTTP 协议
	3	客户端与服务端
	3.1	网页表示语言
	3.2	客户与服务端的任务
	3.3	客户与服务端的软件架构

超文本标记语言 HTML

- 超文本标记语言 HTML 中的 Markup 的意思就是“设置标记”。
- HTML 定义了许多用于排版的命令（即标签）。
- HTML 把各种标签嵌入到万维网的页面中。这样就构成了所谓的 HTML 文档。HTML 文档是一种可以用任何文本编辑器创建的 ASCII 码文件。
- 远程链接：超链的终点是其他网点上的页面。
- 本地链接：超链指向本计算机中的某个文件。

<HTML>

<HEAD>

<TITLE>

text that forms the document title

</TITLE>

</HEAD>

<BODY>

body of the document appears here

</BODY>

</HTML>

动态万维网文档

- 静态文档是指该文档创作完毕后就存放在万维网服务器中，在被用户浏览的过程中，内容不会改变。
- 动态文档是指文档的内容是在浏览器访问万维网服务器时才由应用程序动态创建。

比较项	静态文档	动态文档
内容生成时机	预先创建	请求时实时生成
内容是否固定	同一文件始终相同	因人而异、因时而异
服务器处理	直接读取文件	执行代码、访问数据库
响应速度	快	相对慢（需计算）
适合场景	不常变化的信息	个性化、交互性强的应用

内容纲要

	3	客户端与服务器端
	3.1	网页表示语言
	3.2	客户与服务器端的任务
	3.3	客户与服务器端的软件架构
	4	Web 安全与性能优化

前端与后端

- 前端是指用户直接看到并与之交互的部分。
- 技术组成
 - HTML：页面结构；CSS：页面样式（布局、颜色、字体）
 - JavaScript：页面行为（点击、滑动、动态更新）
 - 框架/库：React、Vue、Angular 等
- 主要任务
 - 展示数据（从后端获取并渲染）
 - 收集用户输入（表单、点击）
 - 与后端通信（通过 HTTP 请求获取或提交数据）
 - 实现交互效果（动画、实时验证）

前端与后端

- 后端是用户看不到但支撑整个应用运行的部分。
- 运行在服务器上，负责处理业务逻辑、数据存储、安全认证等。
- 技术组成
 - 服务器端语言：Java (Spring)、Python (Django/Flask)、Node.js (JavaScript)、PHP、Go 等
 - 数据库：MySQL、PostgreSQL、MongoDB 等
 - 服务器软件：Nginx、Apache
 - API 设计：RESTful、GraphQL

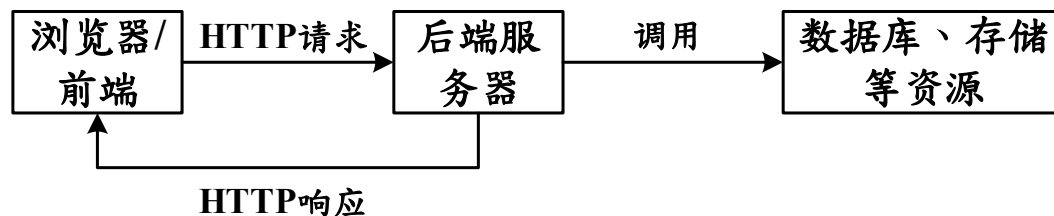
前端与后端

- 后端主要任务

- 接收前端发来的请求
- 执行业务逻辑（如计算价格、验证权限）
- 读写数据库
- 返回响应（HTML 或 JSON 数据）

- 前后端分离是目前主流模式

- 用户请求后，后端返回 JSON 数据，前端通过 JavaScript 将数据渲染成页面。



内容纲要

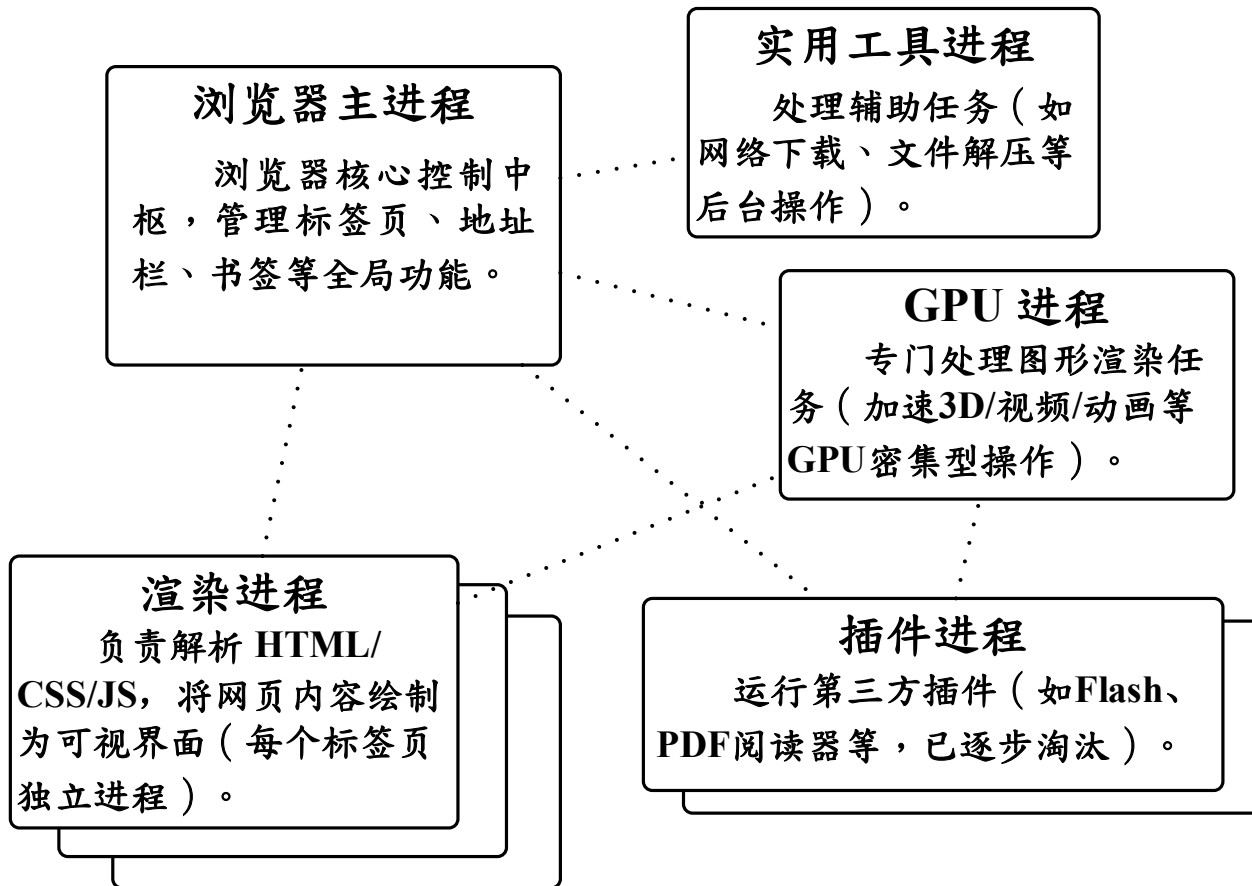
	3	客户端与服务端
	3.1	网页表示语言
	3.2	客户与服务端的任务
	3.3	客户与服务端的软件架构
	4	Web 安全

浏览器架构

- 现代浏览器的多进程架构
 - 浏览器进程
 - 渲染进程
 - GPU进程
 - 网络进程
 - 插件进程
- 多进程设计带来了崩溃隔离、增强安全、性能优化等好处，但也增加了内存占用。
- 加载策略（避免白屏）：同步、异步、推迟。

浏览器架构

• 浏览器的核心模块



服务器架构

- HTTP 服务器软件的组成

- 建立连接：通过 TCP 协议与客户端（如浏览器）建立连接。
- 解析请求：解析客户端发来的 HTTP 请求报文，从中提取出请求方法（GET/POST）、URL、头部信息等内容。
- 生成响应：根据解析出的请求，从服务器上找到对应的资源，生成 HTTP 响应报文（包含状态码、头部和主体），并发送回客户端。

- 常见HTTP服务器

- Nginx、Apache2、IIS

服务器架构（Nginx）

- 事件驱动核心

- 使用操作系统提供的异步 I/O 接口同时管理多个网络连接
- 作用：高并发、低内存、非阻塞
- 工作方式
 - 向操作系统注册一批感兴趣的套接字和事件。
 - 操作系统监测到某个套接字就绪后，通知事件核心。
 - 事件核心调用相应的回调函数（如 HTTP 解析器、静态文件处理器）处理该连接。
 - 处理后继续等待下一轮事件。

服务器架构（Nginx）

- 连接处理器

- 负责建立、管理、关闭与客户端之间的 TCP 连接。
- 工作在事件驱动核心之上，为每个新连接分配必要的资源（内存缓冲区、状态机上下文等）。
- 作用：接受新连接、连接状态管理、超时控制、关闭连接
- 工作方式
 - 当事件核心检测到监听套接字可读时，连接处理器调用 `accept()` 获取新连接的套接字 `fd`（文件描述符），并将其加入事件核心的监控集合中。

服务器架构 (Nginx)

• HTTP 解析器

- 一个状态机，负责将客户端发来的字节流解析成 HTTP 请求报文的结构：请求行、头部字段、空行、消息体。
- 作用：解析请求行、解析头部、分块解码、边界处理、错误检测
- 工作方式：增量解析
 - 每次收到一部分数据就解析一部分
 - 遇到不完整的数据（如头部只发了一半）则保存当前状态，等待更多数据到来后继续解析。
 - 这样避免了需要完整接收整个请求才能开始处理的问题。

服务器架构（Nginx）

- 请求路由器

- 根据解析出的请求 URI 和 HTTP 方法，决定由哪个模块或处理器来生成响应。
- 作用：静态文件路由、动态代理路由、重定向路由、默认处理。
- 工作方式
 - Nginx 的配置文件中使用 location 块来定义路由规则
 - 路由器按优先级（精确匹配 > 前缀匹配 > 正则表达式）匹配请求 URI
 - 然后执行对应的 root、proxy_pass、rewrite 等指令。

服务器架构 (Nginx)

- 静态文件处理器

- 负责将磁盘上的文件读取并作为 HTTP 响应的消息体发送给客户端。
- 作用：路径映射、文件元数据获取（读取文件修改时间、大小等，用于生成头部）、零拷贝发送（直接从系统发送至网卡）、缓存控制、目录索引（访问目录的默认文件）。
- 工作方式
 - 收到静态文件请求后，文件处理器检查文件是否存在、是否有权限。
 - 如果成功，则打开文件，通过 `sendfile()` 发送文件内容；
 - 如果文件不存在，则返回 404。

服务器架构（Nginx）

- 代理模块

- 将客户端发来的 HTTP 请求转发给另一台服务器（称为“上游服务器”），然后将上游服务器返回的响应再传回客户端。Nginx 作为反向代理时，客户端不知道后端服务器的存在。
- 作用：负载均衡、缓存、SSL终止、协议转换、请求/响应改写
- 工作方式
 - 当路由器匹配到 proxy_pass 指令时，代理模块根据配置选择一个上游服务器，建立 TCP 连接或复用已有的 keepalive 连接，将客户端的请求改写后发送过去，然后接收响应并返回给客户端。

内容纲要

	2	HTTP 协议
	3	客户端与服务端
	4	Web 安全
	5	性能优化
	6	前沿技术

Web 安全

- Web 安全的重要性
 - 数据泄露、财产损失、用户信任危机
 - 常见案例：CSDN 密码泄露、Facebook 数据滥用
- 核心安全原则
 - 保密性（ Confidentiality ）：数据不被未授权访问（ 加密 ）
 - 完整性（ Integrity ）：数据不被篡改（ 数字签名、哈希 ）
 - 可用性（ Availability ）：服务持续可用（ 防 DDoS ）
- 安全改进：HTTPS / TLS

Web 常见攻击与防御

攻击类型	原理	防御措施
XSS (跨站脚本)	注入恶意脚本，窃取 Cookie/会话	输入过滤、输出转义、CSP (内容安全策略)、HttpOnly Cookie
CSRF (跨站请求伪造)	伪造用户请求，执行非本意操作	CSRF Token、SameSite Cookie、Referer 校验
SQL 注入	恶意 SQL 语句破坏数据库	参数化查询 (PreparedStatement)、ORM 框架、输入验证
DDoS 攻击	海量请求耗尽服务器资源	限流、CDN 防护、防火墙、负载均衡
中间人攻击 (MITM)	窃听或篡改通信	HTTPS (TLS)、证书校验、HSTS
点击劫持 (Clickjacking)	透明 iframe 诱骗用户点击	X-Frame-Options: DENY、CSP frame-ancestors

代理服务器（ proxy server ）

- 代理服务器（ proxy server ）是一种代表客户端（ 浏览器 ）发出 HTTP 请求的中间服务器。
- 高速缓存将最近请求过的响应暂存在本地磁盘中。
- 工作流程
 - 浏览器与代理服务器建立 TCP 连接，并发送 HTTP 请求。
 - 若代理服务器已缓存所请求的对象，直接返回给浏览器。
 - 若未缓存，代理服务器代表浏览器建立连接并发送请求。
 - 源点服务器返回所请求的对象给代理服务器。
 - 代理服务器将对象复制到本地存储器，然后返回给浏览器。

代理服务器 (proxy server)

- HTTPS 对代理缓存的新挑战
 - 传统代理无法直接查看加密内容，因此不能像处理明文 HTTP 那样随意缓存。
- 内容分发网络 (CDN) 是一种反向代理
- Cache-Control 头部更精细的缓存控制指令
- 缓存安全风险
 - 缓存中毒：攻击者诱使代理缓存恶意内容，污染所有用户。
 - 缓存欺骗：利用 URL 差异绕过缓存策略。

CORS

- 跨域资源共享 (Cross-Origin Resource Sharing , CORS)
 - 一种基于 HTTP 头部的机制，它允许服务器声明哪些来源可以访问自己的资源，从而突破浏览器的同源策略限制。
- 浏览器的同源策略
 - 浏览器出于安全考虑，默认禁止网页向不同源（域名、协议、端口不完全一致）的服务器发起 AJAX/Fetch 请求。
- 解决方案
 - 浏览器自动在请求中添加 Origin 头部，服务器响应时需包含 Access-Control-Allow-Origin 头部
 - 预检请求：OPTIONS 请求

内容纲要

	2	HTTP 协议
	3	客户端与服务端
	4	Web 安全
	5	性能优化
	6	前沿技术

性能优化

- 性能优化的必要性

- 用户体验、网站排名、商业转化率

- 关键性能指标

- 首字节时间、首次内容绘制、最大内容绘制（核心内容）、总阻塞时间（可交互前）、累积布局偏移（视觉稳定性）

- 性能优化的方向

- 网络优化：压缩、CDN、HTTP/2 多路复用、减少请求数
 - 缓存优化：强缓存、协商缓存、Service Worker
 - 渲染优化：关键 CSS 内联、异步加载 JS、懒加载（图片/组件）、减少重排重绘

- 内容分发网络 (Content Delivery Network , CDN)
 - 一个分布在全球各地的服务器集群。
 - 将网站的静态资源缓存到离用户最近的节点上，使用户能够就近获取内容，从而提高访问速度。
- CDN 的工作原理
 - 用户请求一个资源。
 - DNS 解析将域名解析到 CDN 服务商的全局负载均衡系统。
 - 负载均衡系统根据用户 IP、地理位置、节点健康度等，返回最近的边缘节点 IP。
 - 用户向该边缘节点请求资源，获取资源后缓存并返回用户。

内容纲要

	2	HTTP 协议
	3	客户端与服务端
	4	Web 安全
	5	性能优化
	6	前沿技术

前沿技术

- WebSocket

- 只通过一次“握手”，就能建立一条全双工、持久化的通道，之后双方随时都能互发消息，且数据头开销极小
- 为解决HTTP“一问一答”的局限而设计的。

- SSE (Server-Sent Events)

- 一种基于HTTP的单向通信协议，允许服务器主动向客户端推送数据流。
- 特别适合需要服务器持续向浏览器单向推送更新，而浏览器只需被动接收的场景，在资源消耗上比 WebSocket 更低

前沿技术

• WebRTC

- 一项允许浏览器之间直接进行音视频通信或传输数据的开放标准，无需安装任何插件。
- 常用于网页版视频会议、语音通话、共享屏幕，以及远程教育、在线医疗等场景。其核心技术原理允许点对点（P2P）传输，数据直接走最优路径送达对方，延迟更低，配合STUN/TURN服务器还可解决复杂的NAT穿透问题。

• HTTP/3与QUIC

- 使用UDP替代了传统的TCP。它通过多路复用彻底解决了“队头阻塞”问题，建立连接更快，弱网环境表现更优。

谢谢观看



廈門大學
XIAMEN UNIVERSITY



信息学院 黄 焯
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang